
Organizing and Documenting Data Availability

Instructor: Pragati Dahal

DATA SCIENCE FOR THE PUBLIC GOOD · SUMMER 2026



What Are We Building Today?

By the end of this workshop, you will be able to:

- **Identify** which variables exist in your dataset across years
- **Audit** data availability programmatically in R
- **Document** your findings in a structured, shareable format
- **Visualize** the results as an interactive HTML dashboard

Before we start — open the finished dashboard in your browser. That is where we are going.

Where This Fits in Your Project

The problem

Before you clean your data, you need to know what you actually have. If a variable only exists for 7 out of 14 years — you need to know that *before* you spend hours cleaning it.

Today's step

Raw Data → [YOU ARE HERE] → Cleaning → Analysis → Report

The step between downloading data and cleaning it is usually skipped. Today we make it explicit.

tidycensus: Your Data Access Tool

You set up your Census API key in the previous workshop. Now we use it.

`tidycensus` gives you ACS data directly in R — no manual downloading required.

```
1 library(tidycensus)
2 library(tidyverse)
3
4 get_acs(
5   geography = "county",      # unit of analysis
6   variables = "B19013_001",  # which variable
7   state      = "VA",         # which state
8   year       = 2022,         # which year
9   survey     = "acs5"        # 5-year estimates
10 )
```

Finding Variable Codes

ACS variables follow a pattern: Table + Cell number

B19013_001 = Table B19013 (Median Household Income), Cell 001

Use `load_variables()` to search the full list:

```
1 vars <- load_variables(2022, "acs5", cache = TRUE)
2 View(vars)
3
4 # Search by keyword
5 vars |>
6   filter(str_detect(label, "income|Income")) |>
7   select(name, label, concept)
```

Always check the `concept` column — it tells you which table the variable belongs to.

Your First Data Pull

```
1 va_income <- get_acs(  
2   geography = "county",  
3   variables = "B19013_001",  
4   state     = "VA",  
5   year      = 2022,  
6   survey    = "acs5"  
7 )  
8  
9 head(va_income)
```

Output

Each row is one county. Five columns: `GEOID`, `NAME`, `variable`, `estimate`, `moe`

- ▶ **estimate** = the ACS value
- ▶ **moe** = margin of error

Pulling Multiple Years

`map_dfr()` means: run this function for each year, then stack all results into one dataframe.

```
1 years <- 2015:2022
2
3 multi_year <- map_dfr(years, function(yr) {
4   get_acs(
5     geography = "county",
6     variables = c(
7       median_income = "B19013_001",
8       poverty       = "B17001_002"
9     ),
10    state = "VA",
11    year  = yr,
12    survey = "acs5"
13  ) |>
14    mutate(year = yr)
15 })
```

The Problem: Not All Variables Exist

```
1 get_acs(  
2     geography = "county",  
3     variables = "B28002_004", # broadband access  
4     state      = "VA",  
5     year       = 2012,  
6     survey     = "acs5"  
7 )
```

You get an error

ACS did not track broadband subscriptions at the county level before 2017.

This is not a data error — it is measurement history. And you need to know this *before* you build your analysis panel.

The Audit Function

`tryCatch()` lets R handle errors gracefully — instead of crashing, it returns `FALSE`.

```
1 check_variable <- function(var, year) {  
2   tryCatch({  
3     result <- get_acs(  
4       geography = "county",  
5       variables = var,  
6       state     = "VA",  
7       year      = year,  
8       survey    = "acs5"  
9     )  
10    nrow(result) > 0      # TRUE if data exists  
11  }, error = function(e) {  
12    FALSE                 # FALSE if error  
13  })  
14 }  
15  
16 check_variable("B28002_004", 2012) # FALSE  
17 check_variable("B28002_004", 2020) # TRUE
```

Running the Full Audit

```
1 vars <- c(  
2   "B19013_001", "B17001_002", "B15003_022",  
3   "B27001_001", "B01003_001", "B28002_004"  
4 )  
5  
6 years <- 2010:2023  
7  
8 audit_results <- expand_grid(variable = vars, year = years) |>  
9   mutate(available = map2_lgl(variable, year, check_variable))  
10  
11 write_csv(audit_results, "audit_results.csv")
```

This makes ~84 API calls and takes a few minutes. Save immediately with `write_csv()` so you never have to run it again.

What the Audit Reveals

Variable	Coverage	Missing Years
Total Population	✓ 14/14	—
Median Household Income	✓ 14/14	—
Population Below Poverty	✓ 14/14	—
Bachelor’s Degree	⚠ 12/14	2010–2011
Health Insurance Universe	⚠ 12/14	2010–2011
Broadband Access	⚠ 7/14	2010–2016

If your project needs broadband data, your analysis window starts in 2017 — not 2010. This is the kind of thing the audit is designed to surface.

Adding Labels and Categories

```
1 var_metadata <- tibble(  
2   variable = c("B19013_001", "B17001_002", "B15003_022",  
3               "B27001_001", "B01003_001", "B28002_004"),  
4   label     = c("Median Household Income", "Population Below Poverty",  
5               "Bachelor's Degree", "Health Insurance Universe",  
6               "Total Population", "Broadband Access"),  
7   category  = c("Socioeconomic", "Socioeconomic", "Socioeconomic",  
8               "Healthcare Access", "Demographics", "Infrastructure")  
9 )  
10  
11 audit_final <- audit_results |>  
12   left_join(var_metadata, by = "variable") |>  
13   group_by(variable, label, category) |>  
14   summarise(  
15     n_years      = sum(available),  
16     years_present = list(year[available == TRUE]),  
17     missing_years = list(year[available == FALSE]),  
18     .groups = "drop"  
19 )
```

Building the Dashboard

Option 1 — Pure R

```
1 library(reactable)
2 library(htmlwidgets)
3
4 widget <- reactable(
5   audit_final,
6   filterable = TRUE,
7   searchable = TRUE
8 )
9
10 saveWidget(widget,
11   "data_availability.html")
```

Functional, shareable, no AI needed.

Option 2 — R + Claude

```
1 library(jsonlite)
2
3 cat(toJSON(audit_final |>
4   mutate(
5     years_present =
6       map(years_present, as.integer),
7     missing_years =
8       map(missing_years, as.integer)
9   ), pretty = TRUE))
```

Copy JSON → paste into Claude with the prompt on the next slide.

The Claude Prompt

Copy this — then paste your JSON at the bottom

You are a data visualization expert. I have audited variable availability in ACS 5-year estimates for Virginia counties from 2010–2023. Below is a JSON file with the results.

Please create a single self-contained interactive HTML dashboard that shows:

- A grid where rows = variables and columns = years
- Color-coded cells by category (filled = available, grey = missing)
- Hover tooltips showing variable name, category, years present, and missing years
- Filter buttons by coverage (full/partial) and by category
- A summary stats bar at the top

Use Virginia Tech colors: maroon (#861F41), orange (#E5751F), white background. Make it clean, professional, and suitable for a research presentation.

Here is the JSON data: [PASTE YOUR JSON HERE]

The more specific your prompt, the better the output. Mention colors, layout, interactivity — Claude needs that context.

Apply This to Your Own Project

Step 1 — Read your proposal

What variables do you need? Write them down before touching any code.

Step 2 — Find the codes

Use `load_variables()` to find the ACS table and cell for each variable you need.

Step 3 — Run the audit

Which years are actually available? Don't assume — check programmatically.

Step 4 — Document

Add your own labels and categories. Save the audit script — it is part of your project documentation.

Summary

What you learned today

- ▶ How to pull ACS data by variable code using `tidycensus`
- ▶ How to audit variable availability across years with `tryCatch()`
- ▶ Why some variables have gaps — and why that matters for your analysis
- ▶ How to document findings with human-readable labels and categories
- ▶ How to turn your audit into a shareable HTML dashboard

The pipeline

`get_acs()` → `check_variable()` → `expand_grid()` → `reactable` / Claude → HTML

Thank you

Pragati Dahal

Graduate Research Assistant · Agricultural & Applied Economics
Virginia Tech

WORKSHOP MATERIALS AVAILABLE IN THE SHARED DSPG FOLDER

